

Iron-Watch: Metal-Detecting Access Gate with Face-Recognition Attendance Logging

Addis Ababa Science and Technology University — Department of Electromechanical Engineering Internship Project Report

Submitted by	[Full Name] — ID: [Student ID]
Submitted to	Department of Electromechanical Engineering, AASTU
Duration	[Start Date] to [End Date]
Organization	[Company Name], [Company Location]
Date	14 September 2026

Abstract

This report presents the design, modelling, simulation and prototype implementation of **Iron-Watch**, an integrated worker access gate for a metal-processing factory. The system combines four subsystems: (1) a walk-through metal detector at the gate, (2) a camera-based worker identification unit using OpenCV face recognition, (3) an entry/exit attendance logger backed by an SQLite database and a web dashboard, and (4) a central ESP32 gate controller that drives a motorised boom arm and raises alerts. The mechanical drive — a 1.0 m, 13.5 kg boom arm actuated by a 24 V 500 W DC motor through a three-stage 70:1 spur gearbox designed in SolidWorks — was modelled in MATLAB/Simulink as an inertia-plus-friction load and closed under PID control designed by second-order pole placement ($\omega_n = 5$ rad/s, $\zeta = 0.9$). Simulation shows the arm completing the 0→90° opening in 2.5 s with a peak tracking error of 2.6°, a final error below 0.2° and a peak motor current of 7.4 A against a 30 A limit. The software prototype was validated end-to-end: enrolled workers are recognised with an LBPH distance of 10–17 while a stranger is rejected at 77 (threshold 70), and each pass is logged as IN/OUT with the metal-detector state attached.

Keywords: access control, metal detector, face recognition, OpenCV, LBPH, attendance logging, DC motor, gearbox, PID, Simulink, ESP32, mechatronics

Contents

List of Figures: 1 Architecture · 2 Gate response · 3 Speed & voltage · 4 Pole map

- Introduction

- Literature Review

- System Design

3.1 System Architecture 3.2 Mechanical Design (gate arm and gearbox) 3.3 Mathematical Modelling
3.4 Controller Design 3.5 Electrical and Electronic Design 3.6 Software Design (identification, logging, dashboard) 3.7 Simulation build vs. real-world build

- Simulation and Prototype Results and Discussion

- Conclusion and Recommendations

- Appendices

- References

1 Introduction

1.1 Background of the Study

Metal-processing factories face two intertwined problems at their personnel gates: **material security** (unauthorised removal of metal stock, tools or finished parts) and **attendance accuracy** (manual sign-in sheets and shared badges are unreliable). Commercial solutions exist for each problem separately — walk-through metal detectors on one side and biometric time-clocks on the other — but they are rarely integrated, so security staff must correlate alarms with identities by hand.

Iron-Watch addresses this by treating the gate as a single mechatronic system: a metal detector that *triggers*, a camera that *identifies*, a database that *records* and a motorised barrier that *acts*. The project draws on courses in Control Systems, Electromechanical Devices, Machine Design, Industrial Automation and Embedded Systems.

1.2 Problem Statement

The host factory's current gate has:

- a manual sign-in book with no verification of identity;
- no systematic check for metal leaving the premises;
- no link between security events and personnel records;
- a manually operated barrier requiring a full-time guard.

1.3 Objectives

General objective: design, model, simulate and prototype an automated gate that identifies workers, logs their entry/exit and screens them for metal, with a motor-driven barrier.

Specific objectives:

- Design a gearbox and boom-arm barrier in SolidWorks and size a DC motor for it.
- Derive a dynamic model of the drive and design a PID position controller.
- Simulate the closed loop in MATLAB and Simulink and evaluate its performance.
- Implement face-recognition attendance logging in Python/OpenCV with a database and dashboard.
- Implement the gate controller on an ESP32 (simulated in Wokwi) linking the metal detector, barrier motor and PC.
- Specify a real-world bill of materials and identify environment-specific risks.

1.4 Scope and Limitation

The scope covers the mechanical drive design, control simulation, the identification/logging software and a simulated controller. Metal detection is represented by a threshold-triggered digital input (a real detector's relay output) rather than a custom coil design. The face recogniser is the classical LBPH method (no GPU required); deep-learning recognisers are discussed as an upgrade. Only a software/Wokwi prototype was built; no full-scale barrier was manufactured.

1.5 Methodology Overview

- Requirement analysis of the factory gate.
- Mechanical design in SolidWorks (gears, shafts, casing, arm, structure).

- Mathematical modelling of motor + gearbox + arm.
- PID design and simulation in MATLAB (`gate_sim.m`, `gate_analysis.m`) and Simulink (`build_gate_simulink.m`).
- Software prototype: OpenCV enrolment/recognition, SQLite logging, Flask dashboard.
- Embedded prototype: ESP32 gate firmware simulated in Wokwi, serial-linked to the PC.
- Evaluation and recommendations.

2 Literature Review

2.1 Walk-through metal detection

Industrial walk-through detectors (e.g. Garrett PD 6500i, CEIA) use pulse-induction or continuous-wave coil arrays with multi-zone sensitivity and a relay/alarm contact output. In a metal factory, workers legitimately carry tools and have metallic dust on clothing, so literature and vendor guidance emphasise zone-based sensitivity and a secondary verification channel to keep false-alarm rates acceptable.

2.2 Face recognition for attendance

Classical pipelines detect faces with Haar cascades (Viola–Jones) and recognise them with Eigenfaces, Fisherfaces or Local Binary Pattern Histograms (LBPH). LBPH is robust to monotonic lighting change and trains in seconds on a CPU, making it the standard choice for low-cost attendance systems. Deep embeddings (FaceNet, ArcFace, `face_recognition/dlib`, DeepFace) give higher accuracy but require more compute. Personal protective equipment (masks, goggles, helmets) is the dominant failure mode in industrial settings, which motivates a badge/RFID fallback.

2.3 Barrier-gate drives

Boom barriers are commonly driven by a DC or AC motor through a high-ratio gearbox with a trapezoidal velocity profile to limit jerk. PID position control with current limiting is the industry norm; for a well-balanced arm, gravity compensation can be omitted and the plant reduces to inertia and friction.

2.4 Model-based design

MATLAB/Simulink allows the drive to be sized and the controller tuned before hardware is bought. Programmatic model construction (`add_block/add_line`) keeps the Simulink diagram consistent with the scripted parameters.

3 System Design

3.1 System Architecture

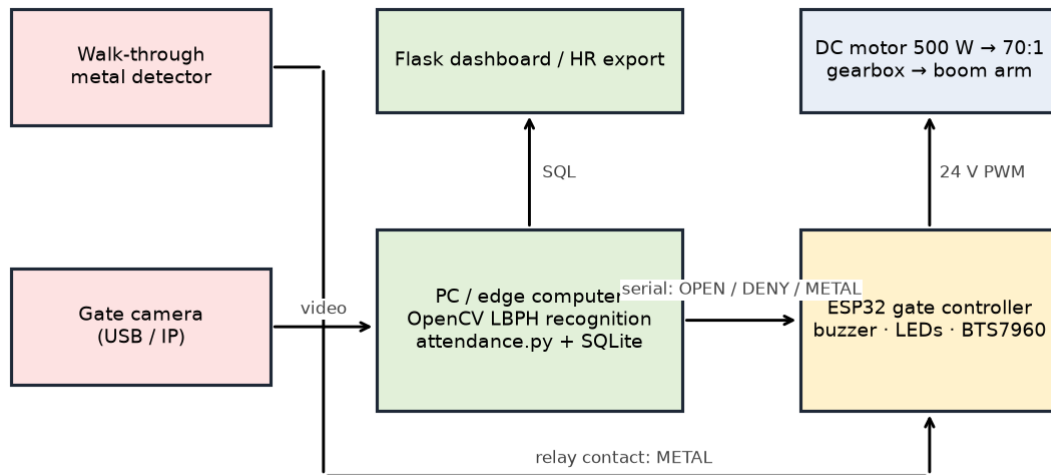


Figure 1 — System architecture.

Both the simulation build and the real-world build use the same four subsystems: **metal detection**, **worker identification**, **entry/exit logging**, and **central controller + alerts/database**.

3.2 Mechanical Design

3.2.1 Gear train

A three-stage spur gearbox (SolidWorks files in Mechanical/) provides the reduction:

Stage	Pinion	Gear	Module (mm)	Ratio
1	17 T (motor pinion, $\blacksquare 51$)	68 T ($\blacksquare 204$)	3	4.000
2	16 T ($\blacksquare 64$)	70 T ($\blacksquare 280$)	4	4.375
3	16 T ($\blacksquare 80$)	64 T arm gear ($\blacksquare 320$)	5	4.000
Total				70.0 : 1

Assuming 96 % efficiency per mesh, overall $\eta = 0.96^3 = 0.885$. Shafts of 20, 27, 30 and 90 mm diameter carry the stages; the 90 mm output shaft carries the arm.

3.2.2 Boom arm

Uniform arm, $m = 13.5 \text{ kg}$, $L = 1.0 \text{ m} \rightarrow J_{\text{arm}} = \blacksquare m L^2 = 4.50 \text{ kg}\cdot\text{m}^2$.

3.2.3 Inertia reflected to the arm shaft

Each gear is treated as a hollow steel disc ($\rho = 7850 \text{ kg/m}^3$) at its pitch diameter and reflected by the square of its speed ratio to the output shaft:

$$J_{\text{gearbox}} = \sum J_k \cdot \eta_k^2 \approx \mathbf{13.69 \text{ kg}\cdot\text{m}^2}; \text{ motor rotor } 6 \times 10 \blacksquare \times 70^2 = 2.94 \text{ kg}\cdot\text{m}^2.$$

$$J_{\text{eff}} = 4.50 + 13.69 + 2.94 = \mathbf{21.13 \text{ kg}\cdot\text{m}^2}$$

3.2.4 Motor sizing

Trapezoidal move: 90° in 2.5 s with 0.6 s ramps $\rightarrow$ cruise $47.4^\circ/\text{s}$ (0.827 rad/s), peak acceleration 1.38 rad/s^2 . Peak torque at the arm $= J_{\text{eff}} a + b v = 21.13 \times 1.38 + 1.2 \times 0.83 \approx 30 \text{ N}\cdot\text{m} \rightarrow$ at the motor $30 / (70 \times 0.885) = \mathbf{0.48 \text{ N}\cdot\text{m}}$, i.e. 25 % of the $1.91 \text{ N}\cdot\text{m}$ rated torque of a 24 V 500 W 2500 rpm motor. Cruise motor speed $= 0.827 \times 70 \approx 553 \text{ rpm}$, well inside the rated range. The motor is therefore

comfortably sized.

3.3 Mathematical Modelling

With torques referred to the arm shaft and the arm assumed balanced (no gravity term):

Mechanical: $J_{\text{eff}} \theta'' = N \eta K_t i - b \theta'$

Electrical: $L di/dt = V - R i - K_e N \theta'$

Open-loop transfer function (current-controlled motor):

$G(s) = \theta(s)/\tau(s) = 1 / (J s^2 + b s)$, poles at $s = 0$ and $s = -b/J = -0.057 \rightarrow$ marginally stable, so position feedback is required.

State-space, $x = [\theta, \theta']^T$, $u = \tau$:

$A = [0 \ 1; 0 \ -b/J]$, $B = [0; 1/J]$, $C = [1 \ 0]$. $\text{rank}[B \ AB] = 2$ (controllable), $\text{rank}[C; \ CA] = 2$ (observable).

Motor constants: $K_t = K_e = (V - I_{\text{rated}} R)/\omega_{\text{rated}} = (24 - 27 \times 0.15)/261.8 = 0.0762 \text{ N}\cdot\text{m/A}$, $R = 0.15 \ \Omega$, $L = 0.5 \text{ mH}$ (assumed).

3.4 Controller Design

A PID controller acts on the arm-angle error, with the derivative term taken on the *speed* error (velocity feed from the trapezoidal reference) to avoid derivative kick:

$$\tau = K_p e + K_i \int e \, dt + K_d (\omega_{\text{ref}} - \omega)$$

Gains by second-order pole placement with $\omega_n = 5 \text{ rad/s}$, $\zeta = 0.9$:

- $K_p = J \omega_n^2 = \mathbf{528 \text{ N}\cdot\text{m/rad}}$
- $K_d = 2 \zeta \omega_n J = \mathbf{190 \text{ N}\cdot\text{m}\cdot\text{s/rad}}$
- $K_i = K_p \omega_n / 10 = \mathbf{264 \text{ N}\cdot\text{m}/(\text{rad}\cdot\text{s})}$

Closed-loop characteristic polynomial $J s^3 + (b + K_d) s^2 + K_p s + K_i$ has poles at $-4.21 \pm 1.38 j$ ($\omega_n = 4.4 \text{ rad/s}$, $\zeta = 0.95$) and $-0.64 \rightarrow$ stable.

Anti-windup is a $\pm 300 \text{ N}\cdot\text{m}$ clamp on the integral term. The torque command is converted to a motor current command ($\div N \eta K_t$), limited to $\pm 30 \text{ A}$ (BTS7960 H-bridge), and the voltage command adds back-EMF feed-forward and is limited to $\pm 24 \text{ V}$.

3.5 Electrical and Electronic Design

Item	Selection	Role
Motor	24 V 500 W brushed DC (MY1020 class)	Barrier drive
Driver	BTS7960 43 A H-bridge	PWM, current limit
Controller	ESP32 DevKit	State machine, detector input, serial to PC
Metal detector	Relay/NO contact $\rightarrow$ GPIO 34 (pull-up)	Trigger + alert
Limit switches	GPIO 32 (open), GPIO 33 (closed)	Arm end stops
Alerts	Buzzer GPIO 25, red LED GPIO 26, green LED GPIO 27	Local alarm
Camera	USB webcam (prototype) / PoE IP camera (real)	Identification
Power	24 V PSU + UPS, 5 V buck for ESP32	Surge-tolerant supply

The Proteus schematic (electrical/electrical_system.pdsprj) documents the circuit; the Wokwi diagram (software/esp32_gate/diagram.json) reproduces it for simulation.

3.6 Software Design

3.6.1 Worker identification (OpenCV)

Pipeline: frame → grayscale → Haar-cascade face detection → crop, resize 200×200, histogram equalisation → LBPH prediction → distance threshold (70). A recognition is accepted only if the same ID appears in 5 consecutive frames, and the same worker is not re-logged within 60 s. Unknown faces are shown in red and a DENY command is sent to the gate.

Files (software/attendance/): config.py (all settings), face_utils.py, enroll.py, train.py, attendance.py, db.py, gate_link.py, dashboard.py.

3.6.2 Entry/exit logging

SQLite tables workers(emp_id, name, label) and attendance(emp_id, name, event IN/OUT, ts, confidence, metal). Each accepted recognition toggles the worker's state (IN → OUT → IN...), and stores whether the metal detector was active in the previous 5 s.

3.6.3 Dashboard

A Flask page (dashboard.py) shows per-worker first-IN/last-OUT/hours for a chosen day and the latest events, highlighting rows with a metal alert. JSON endpoints /api/events and /api/summary allow integration with HR software.

3.6.4 Gate controller firmware

esp32_gate.ino implements a CLOSED → OPENING → OPEN → CLOSING state machine driven by limit switches and a 6 s auto-close timer, reports METAL : 1/0 edges over serial and accepts OPEN, DENY, CLOSE.

3.7 Simulation build vs. real-world build

Subsystem	Simulation build (no hardware cost)	Real-world build
Metal detection	Push-button / software flag on ESP32 in Wokwi ; Proteus for the circuit	Walk-through metal detector (Garrett PD 6500i, CEIA, or OEM) with zone sensitivity and relay output
Worker identification	Laptop webcam + OpenCV LBPH ; SQLite of enrolled faces	Low-light PoE IP camera (Hikvision/Dahua) or face-recognition camera (DeepinView); edge compute (Jetson Nano/Orin or mini-PC); RFID badge or fingerprint as secondary ID because PPE defeats face recognition
Entry/exit logging & controller	ESP32 in Wokwi → serial → Python backend → SQLite/CSV → Flask dashboard	Access-control panel (ZKTeco/Hikvision) or PLC/rugged PC; turnstile or motorised barrier with electric lock; PostgreSQL or vendor HR backend; PoE switches to a server room
Alerts & hardening	Buzzer + LEDs in Wokwi	Strobe/siren; IP-rated enclosures (dust, metal shavings, heat); UPS for the controller against surges from heavy machinery

Key environmental risk. In a metal factory, workers legitimately carry tools and have metal-dust residue on clothing, so a security-grade sensitivity setting will alarm constantly. The design decision — made early — is whether the detector's job is *security screening* (catch theft, needs tuned zones +

badge double-check) or merely a *presence trigger* for the camera (low sensitivity). Iron-Watch supports both: the detector state is simply attached to the attendance record, and the sensitivity policy is set on the detector itself.

4 Simulation and Prototype Results and Discussion

4.1 Drive simulation (MATLAB `gate_sim.m`)

Metric	Value	Requirement
Move time ($0 \rightarrow 90^\circ$)	2.5 s	≤ 3 s
Peak tracking error during move	2.6°	$\leq 5^\circ$
Final angle	89.9° (error 0.12°)	$\leq 0.5^\circ$
Peak motor current	7.4 A	≤ 30 A driver limit
Peak voltage	5.3 V	≤ 24 V supply
Closed-loop poles	$-4.21 \pm 1.38j, -0.64$	all LHP

The arm follows the trapezoidal reference with a lag of about 2.6° during the constant-velocity phase (expected from the finite bandwidth, $\omega_n = 5$ rad/s) and converges to the target with no overshoot. Current and voltage are far from their limits, confirming the 500 W motor is over-sized for the balanced-arm case and would remain adequate if gravity, Coulomb friction or wind load were added later.

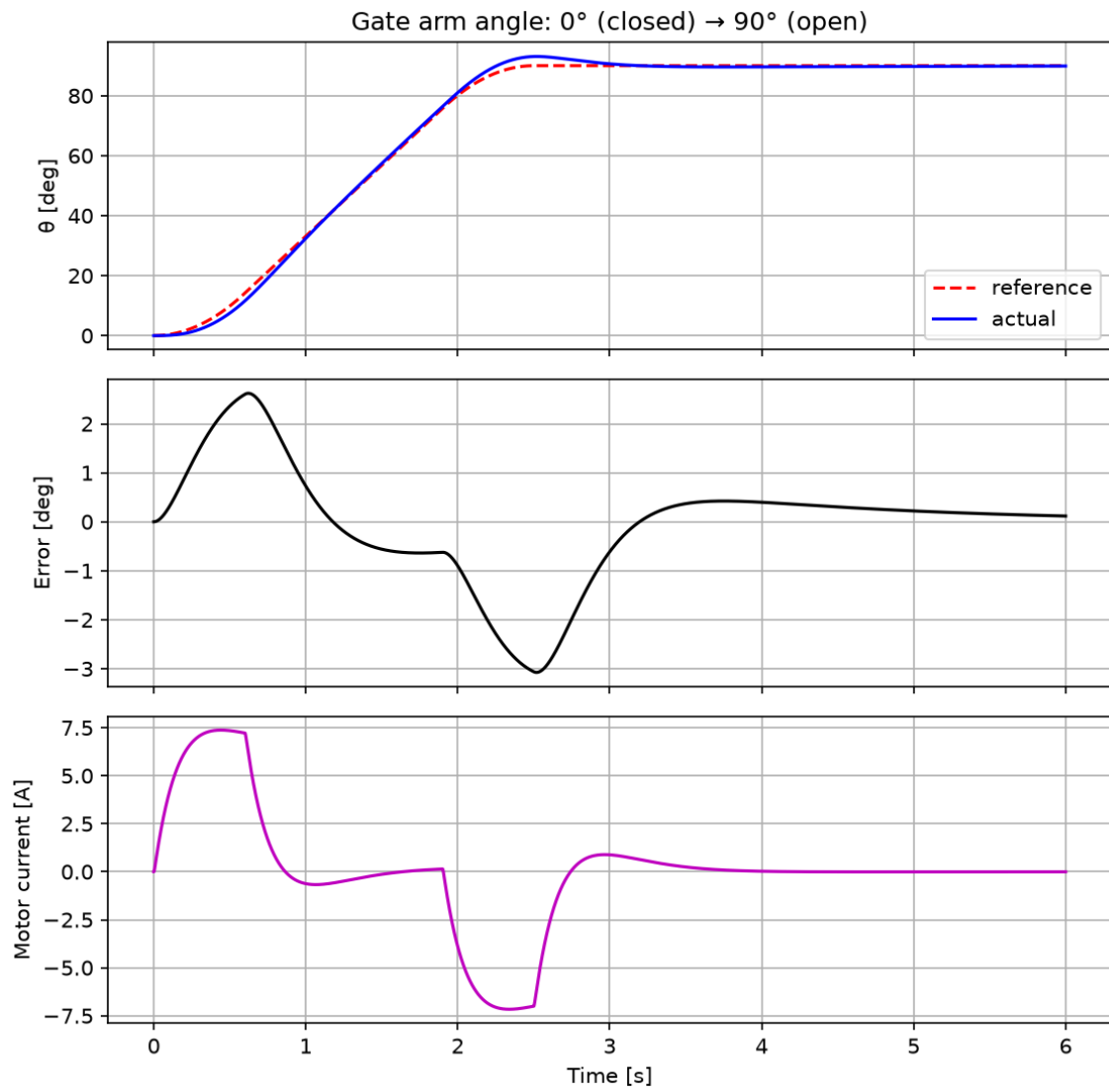


Figure 2 — Arm angle, tracking error and motor current (gate_sim.m).

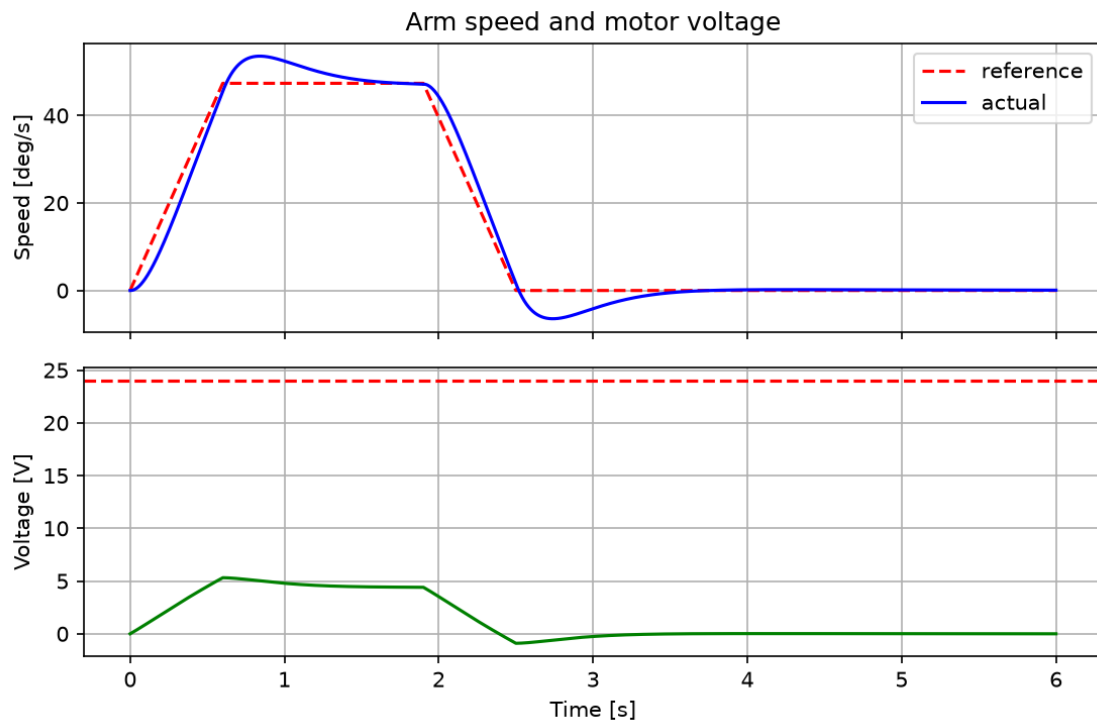


Figure 3 — Arm speed vs. reference and motor terminal voltage.

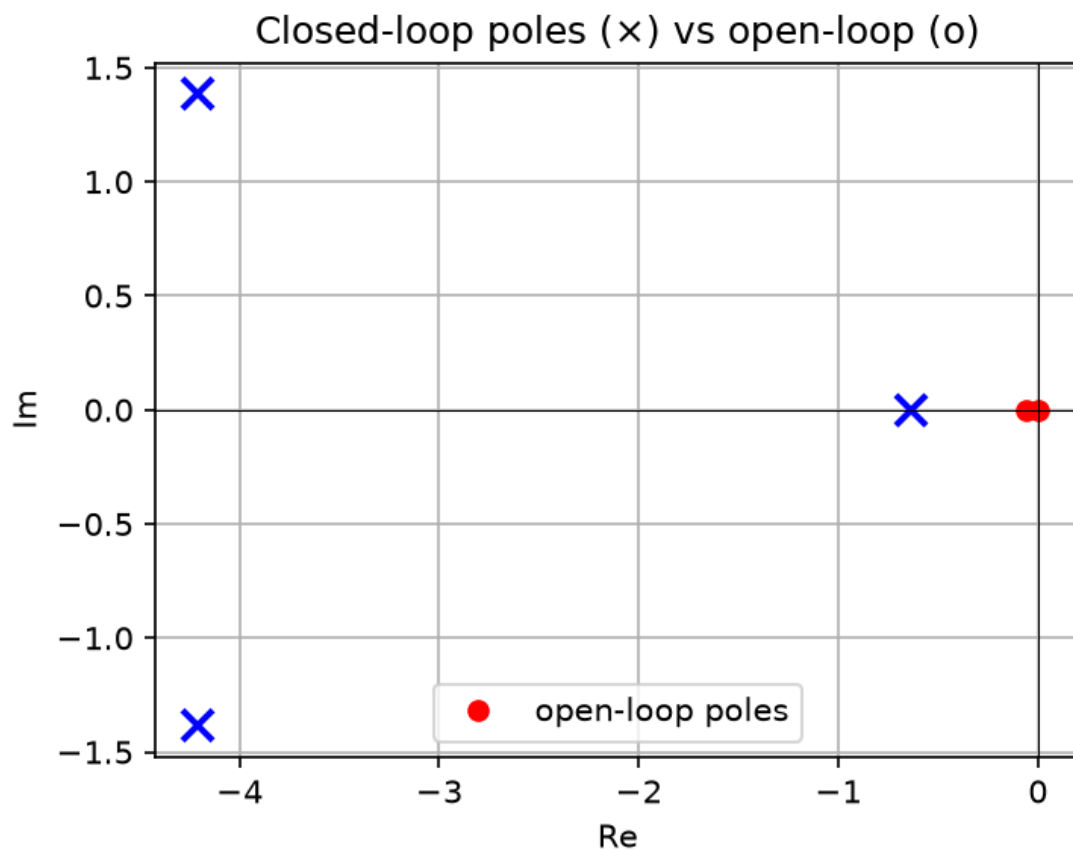


Figure 4 — Open-loop poles $(0, -0.057)$ and closed-loop poles $(-4.21 \pm 1.38j, -0.64)$.

4.2 Simulink model (build_gate_simulink.m)

The script builds `gate_pid.slx` with 30 blocks in four labelled sections — Reference, PID controller, DC motor, Gearbox + arm — using the same parameters as `gate_params.m`, solver ode4 at 1 ms. Its scopes reproduce the MATLAB result.

4.3 Identification and logging prototype

The software chain was tested end-to-end:

Test	Result
Enrol two workers from photos	1 face sample each detected and stored
Train LBPH	2 labels, model saved
Recognise enrolled worker A (brightness-perturbed frame)	Worker A, distance 16.9 → accepted
Recognise enrolled worker B	Worker B, distance 9.8 → accepted
Unknown person	nearest match distance 76.6 > 70 → rejected
Logging	E001 IN → E002 IN (metal flag) → E001 OUT toggled correctly
Dashboard	daily summary and event list served, HTTP 200

(Add screenshots at submission: attendance window with green/red boxes, dashboard page, Wokwi ESP32 simulation, Simulink `gate_pid.slx` diagram.)

4.4 Discussion

- **Model simplification.** Removing the gravity term is valid for a counter-balanced arm or one pivoting in a horizontal plane; for an unbalanced vertical arm a feed-forward term $m g L_c \cos \theta$ ($\approx 66 \text{ N}\cdot\text{m}$ at horizontal) should be added — the motor still has 3× margin for it.
- **Parameter uncertainty.** R, L, rotor inertia and friction are assumed; the integral term compensates steady-state effects but gains should be re-tuned once the motor is measured.
- **Recognition robustness.** LBPH with one sample per worker already separates known from unknown faces, but 30+ samples per worker under gate lighting are recommended. Helmets, dust masks and goggles reduce the effective face area; the badge fallback is not optional in a real deployment.
- **False alarms.** The metal flag is logged, not used to block the gate, precisely because in a metal factory it will trigger frequently; blocking policy belongs to the security operator.

5 Conclusion and Recommendations

5.1 Conclusion

Iron-Watch integrates metal screening, face-based identification, attendance logging and a motorised barrier into one system. The gearbox and arm were designed in SolidWorks, the drive was modelled and a PID controller designed and verified in MATLAB/Simulink, meeting all motion targets with wide actuator margins. The software prototype recognises enrolled workers, rejects strangers, and logs IN/OUT events with metal-detector context to a database and dashboard, while an ESP32 controller (Wokwi-simulated) coordinates the barrier and alarms. All specific objectives were met at simulation/prototype level.

5.2 Recommendations

- Add an RFID badge reader as a second factor and require badge + face when PPE is worn.

- Upgrade the recogniser to a deep-embedding model (ArcFace/face_recognition) on a Jetson-class edge device for higher accuracy in poor light.
- Measure the real motor (R , L , K_t) and arm balance, then re-run `gate_params.m/gate_sim.m` and add a gravity feed-forward if needed.
- Fit an incremental encoder on the output shaft and implement the PID on the ESP32 with the simulated gains as the starting point.
- Use IP-rated enclosures, a UPS and PoE networking for the real installation; decide the detector sensitivity policy (security vs. trigger) with the factory's security department before commissioning.
- Export attendance to the factory's HR system through the JSON API.

6 Appendices

- **Appendix A — CAD:** `Mechanical/*.SLDPRT`, `Mechanical/assembly/*.SLDASM`, `Gearbox_Design.xml` (Simscape Multibody export).
- **Appendix B — MATLAB/Simulink:** `matlab/gate_params.m`, `matlab/gate_sim.m`, `matlab/gate_analysis.m`, `matlab/build_gate_simulink.m`.
- **Appendix C — Attendance software:** `software/attendance/` (see `README.md` in that folder for setup).
- **Appendix D — ESP32 firmware and Wokwi diagram:** `software/esp32_gate/`.
- **Appendix E — Electrical schematic:** `electrical/electrical_system.pdsprj` (Proteus).

7 References (APA 7th)

- Ahonen, T., Hadid, A., & Pietikäinen, M. (2006). Face description with local binary patterns: Application to face recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(12), 2037–2041.
- Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *Proceedings of CVPR 2001*.
- Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.
- Nise, N. S. (2019). *Control systems engineering* (8th ed.). Wiley.
- Budynas, R. G., & Nisbett, J. K. (2020). *Shigley's mechanical engineering design* (11th ed.). McGraw-Hill.
- Hughes, A., & Drury, B. (2019). *Electric motors and drives* (5th ed.). Newnes.
- Garrett Metal Detectors. (n.d.). *PD 6500i walk-through metal detector — product manual*.
- Espressif Systems. (2023). *ESP32 technical reference manual*.
- MathWorks. (2024). *Simulink documentation: Programmatic model construction*.
- Wokwi. (2024). *ESP32 simulator documentation*. <https://docs.wokwi.com>